

Proposta de Projecto

Streaming Recommendation Graph Engine

***Aplicação de Estruturas de dados não lineares e Programação
Orientada aos Objetos (POO)***

Linguagens Programação II

Rui Silva Moreira

rmoreira@ufp.edu.pt

João Viana

jviana@ufp.edu.pt

Algoritmos Estruturas Dados II

José Torres

jtorres@ufp.edu.pt

Março 2026

Universidade Fernando Pessoa

Faculdade de Ciência e Tecnologia

1. Definição do Problema

Neste projeto pretende-se que os alunos modelem, implementem, testem e documentem uma aplicação Java standalone que simule o núcleo de um sistema de utilização e recomendação de uma plataforma de streaming, como Netflix, HBO ou Prime. O sistema deverá representar utilizadores/clientes do sistema, conteúdos multimédia (e.g. filmes, séries, documentários, etc.), artistas (e.g. atores, realizadores) e géneros de conteúdos, recorrendo a estruturas de dados não lineares e a uma abordagem orientada aos objetos.

Numa primeira fase, tanto os conteúdos multimédia como os utilizadores e artistas deverão ser identificados por um ID único, bem como conter toda informação relevante para caracterizar essas entidades. Todas as entidades deverão ser armazenadas/organizadas em estruturas adequadas, do tipo Symbol Table (e.g., HashMap, Red-Black BST, etc.).

Numa segunda fase, deverão utilizar-se grafos $G=(V,E)$ para representar relações entre as entidades (e.g., utilizador visualiza conteúdo, utilizador classifica filme, ator participou em filme, filme possui realizador, utilizador segue utilizador, etc.). A representação em grafo deverá ser pesada e direcionada, para permitir organizar as diversas relações (edges - E) entre as entidades (vértices/nodes - V) relevantes da plataforma de streaming. O grafo deverá ser heterogéneo (permitir vários tipos de vértices) e multi-relacional (permitir vários tipos de pesos nas arestas entre cada par de vértices). Por exemplo, o grafo deverá permitir representar as visualizações dos conteúdos por cada utilizador, as classificações de cada utilizador para cada conteúdo, as relações entre os artistas e os conteúdos, etc. As interações dos utilizadores com os conteúdos devem possuir também timestamps que os permitam organizar temporalmente (e.g. data de visualização de um filme, data em que um artista realizou ou foi actor num filme).

O sistema deverá permitir a representação e armazenamento de toda a informação necessária, bem como a execução de algoritmos de manipulação e análise dos dados representados. Por exemplo, relativamente à representação do grafo, deverá incluir o cálculo de caminhos/relações mais curtas, análises de conexão, extração de subgrafos e geração de recomendações simples baseadas em proximidade ou relações estruturais.

A solução deverá funcionar num único computador (arquitetura standalone), devendo utilizar-se a biblioteca algs4 (cf. Sedgwick - Princeton), nomeadamente para as estruturas de dados e algoritmos propostos. Por exemplo, as entidades poderão ser representadas como vértices do grafo recorrendo a herança ou composição de estruturas de dados base da package algs4 (e.g. DirectWeightedGraph, SymbolGraph, DirectedEdge, etc.). Desta forma

não terão que implementar as estruturas de dados básicas, podendo concentrar-se na lógica e requisitos funcionais da aplicação proposta.

Pretende-se que organizem o trabalho em *packages* de classes para as várias entidades e funcionalidades úteis à implementação do problema proposto, de modo a satisfazer os requisitos propostos. Pretende-se ainda que implementem casos de teste (*Test Cases*) organizados pelas mesmas *packages* e respetivos requisitos. As funções de teste devem ser responsáveis por chamar todos os métodos necessários para demonstrar a correta funcionalidade de cada um dos requisitos isoladamente. Cada caso de teste deverá ser devidamente documentado numa função *static*, responsável por criar todo o contexto de input a utilizar no teste (preferencialmente provenientes de ficheiros ou definidos programaticamente nas funções de teste), bem como pelos valores de output/resultado do teste enviados para a consola e/ou escritos em ficheiros.

Estão previstas duas fases de desenvolvimento, com **apenas uma apresentação/defesa final presencial** (“viva voz”), a acontecer no final do semestre, em datas a anunciar pelos docentes. A **defesa será de presença obrigatória para todos os elementos de cada grupo. Todos os membros do grupo devem estar preparados para responder a quaisquer perguntas colocadas pelos docentes durante a defesa, independentemente da distribuição de tarefas previamente estabelecida entre os elementos do grupo.** Os projetos entregues fora do prazo ou que não sejam apresentados de “viva voz”, não serão considerados para classificação.

2. Requisitos Funcionais

Fase 1: BD + API de Consulta e Gestão de Entidades

R1. Modelizar em diagramas de classes UML todas as entidades do domínio e respectivas associações (NB: a representação das entidades e suas relações numa estrutura em grafo, poderá ser considerada apenas na Fase 2).

R2. Deverão utilizar pelo menos uma *Symbol Table* (ST) para armazenar informação cuja chave (*key*) não necessite ser ordenável. Devem existir funções para **inserir, remover, editar e listar toda a informação**, para cada uma das STs consideradas na base de dados e disponibilizar a API respectiva.

R3. Utilizar pelo menos uma BST balanceada (e.g. Red-Black) para gerir dados ordenados e respectivas pesquisas e operações para inserir, remover, editar e listar informação (e.g. gerir entidades com datas associadas) e disponibilizar a API respectiva. Por exemplo:

- a) Vários tipos de pesquisas de todos os utilizadores de uma dada região e/ou data de registo no sistema e/ou preferências de conteúdos, etc.;
- b) Vários tipos de pesquisas de utilizadores por substring no nome e de uma dada região e/ou data de registo no sistema e/ou preferências de conteúdos, etc.;
- c) Vários tipos de pesquisas de todos os artistas de certas nacionalidades e/ou género e/ou com idades compreendidas entre duas datas, etc.;
- d) Vários tipos de pesquisas de artistas por substring no nome e de certas nacionalidades e/ou género e/ou com idades compreendidas entre duas datas, etc.;
- e) Vários tipos de pesquisas de todos os conteúdos por tipo, género e/ou data de publicação, etc.;
- f) Vários tipos de pesquisas de conteúdos por substring no título, tipo e/ou género e/ou data de publicação, etc.;
- g) Vários tipos de pesquisas de conteúdos com diferentes critérios (e.g. género, duração, ano, etc.).

R4. Implementar mecanismos de validação de consistência entre as estruturas de dados. Por exemplo, garantir que a mesma entidade é utilizada nas várias estruturas em que é utilizada/inserida. A remoção de uma entidade da estrutura principal deve implicar a sua remoção das restantes estruturas, podendo ser necessário arquivar a informação (e.g. ao remover um utilizador da base de dados deve garantir-se a sua remoção total do sistema e respectivo arquivamento em ficheiro ou em estruturas auxiliares).

R5. Deverão criar unidades de teste, recorrendo a classes e funções *static*, para validar todas as estruturas e funcionalidades pretendidas. As funções devem popular as diversas STs da aplicação para a realização dos testes e apresentar os resultados na consola para todos os requisitos solicitados (idem para Fase 2).

R6. Todas as classes devem ser devidamente comentadas e anotadas para ser possível gerar a respectiva documentação, via **Javadoc**, incluindo todos os atributos e funções implementadas (idem para Fase 2).

Fase 2: Grafo + API para Gestão de Relações entre Entidades

R7. Representar as entidades e respectivas relações através de um grafo pesado direcionado (e.g. representar relações entre utilizadores e conteúdos). Disponibilizar a API respectiva.

R8. Utilizar a representação em grafo para implementar diversas funcionalidades:

- a) Calcular diferentes caminhos mais curtos entre dois utilizadores com base em diversos atributos : visualização de filmes (tempo ou true/false); visualização de séries (tempo, episódios ou true/false); utilizadores que seguem utilizadores;

considerando apenas subconjuntos de conteúdos, utilizadores e artistas (subgrafo). Calcular caminhos mais curtos entre dois artistas com base em conteúdos onde participaram.

- b) Extrair e analisar subgrafos de relações entre utilizadores ou entre géneros de conteúdos e respetivos utilizadores (eg. subgrafo com conteúdos de apenas um género ou disponíveis apenas numa determinada região, subgrafo com utilizadores que classificaram conteúdos acima de x).
- c) Determinar se um grafo ou subgrafo é conexo.
- d) Gerar recomendações simples baseadas em proximidade estrutural.
- e) Pesquisar as estatísticas de visualização de um documentário entre duas datas;
- f) Pesquisar todos os utilizadores que viram séries de um determinado género num determinado período;
- g) Pesquisar todos os seguidores de um utilizador que viram o mesmo filme num determinado intervalo.

R9. Desenvolver uma interface gráfica (GUI) para: a) inserir/editar entidades STs e pesquisas; b) visualizar relações do grafo; c) visualizar pesquisas efetuadas.

R10. Permitir importação/exportação de dados a partir de ficheiros de texto. Todos os dados referentes à aplicação (e.g. utilizadores, etc.) e respectivas pesquisas deverão ser guardadas em ficheiros de texto para consulta posterior. Deverá ser ainda possível popular as diversas estruturas de dados (cf. Grafo e STs relacionadas) da aplicação com o conteúdo dos ficheiros de texto de entrada (carregar a informação a partir de ficheiros txt)

R11. Permitir serialização binária do estado completo do sistema a partir de ficheiros binários, isto é, exportar e importar dados (e.g. Grafo e STs relacionadas) com recurso a ficheiros binários.

Cotação dos Requisitos

	Fase 1						Fase 2				
REQ	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	R11
AED2	0	2,5	3,5	1,5	1,5	1	4	3	0	2	1
LP2	1	3	3	1	1	1	3,5	2,5	1,5	1,5	1

(NB: cotação 0-20)

3. Entrega de documentos

O projeto proposto deve ter uma implementação orientada aos objetos em Java. Todo o código (algoritmos e estruturas de dados) deve ser complementado com comentários apropriados, que facilitem a compreensão do mesmo e a respectiva geração automática de documentação.

Deve ser incluída uma explicação dos algoritmos implementados e uma menção ao desempenho dos mesmos assim como dos testes efetuados/implementados num ficheiro **README.md** escrito em markdown language (<https://www.markdownguide.org/>) e localizado na raiz do seu repositório do projeto. Devem ainda ser realizados testes unitários que demonstrem o bom funcionamento das classes desenvolvidas.

Todos os elementos necessários à avaliação deverão estar disponíveis através de um repositório **git** (<https://github.com/>) partilhado com os docentes desde o início do trabalho. **Todos os elementos do grupo devem colaborar com a atualização do repositório git, de modo contínuo e ao longo do semestre, e usando contas institucionais de email da UFP.** No git, o **nome do repositório** escolhido deve codificar os números dos alunos, as UCs e ano letivo, como por exemplo: **1234_5678_aed2_lp2_202425**. Este repositório deve incluir os seguintes componentes complementares:

- i) **Modelos de classes** (ficheiros **zargo** e **PNG** dos diagramas de classes);
- ii) **Código fonte** de toda a implementação do projecto;
- iii) **Ficheiros de teste** contendo os respectivos dados *input/output* utilizados;
- iv) **Documentação** do código (páginas HTML geradas com *JavaDoc*).

Links para consulta

Devem utilizar a biblioteca **algs4**, desenvolvida por Robert Sedgewick and Kevin Wayne, da Universidade de Pinceton, e disponível em <https://algs4.cs.princeton.edu/home>